TIL

エージェント内部構造

LLM-Agent-Controller

[概要]

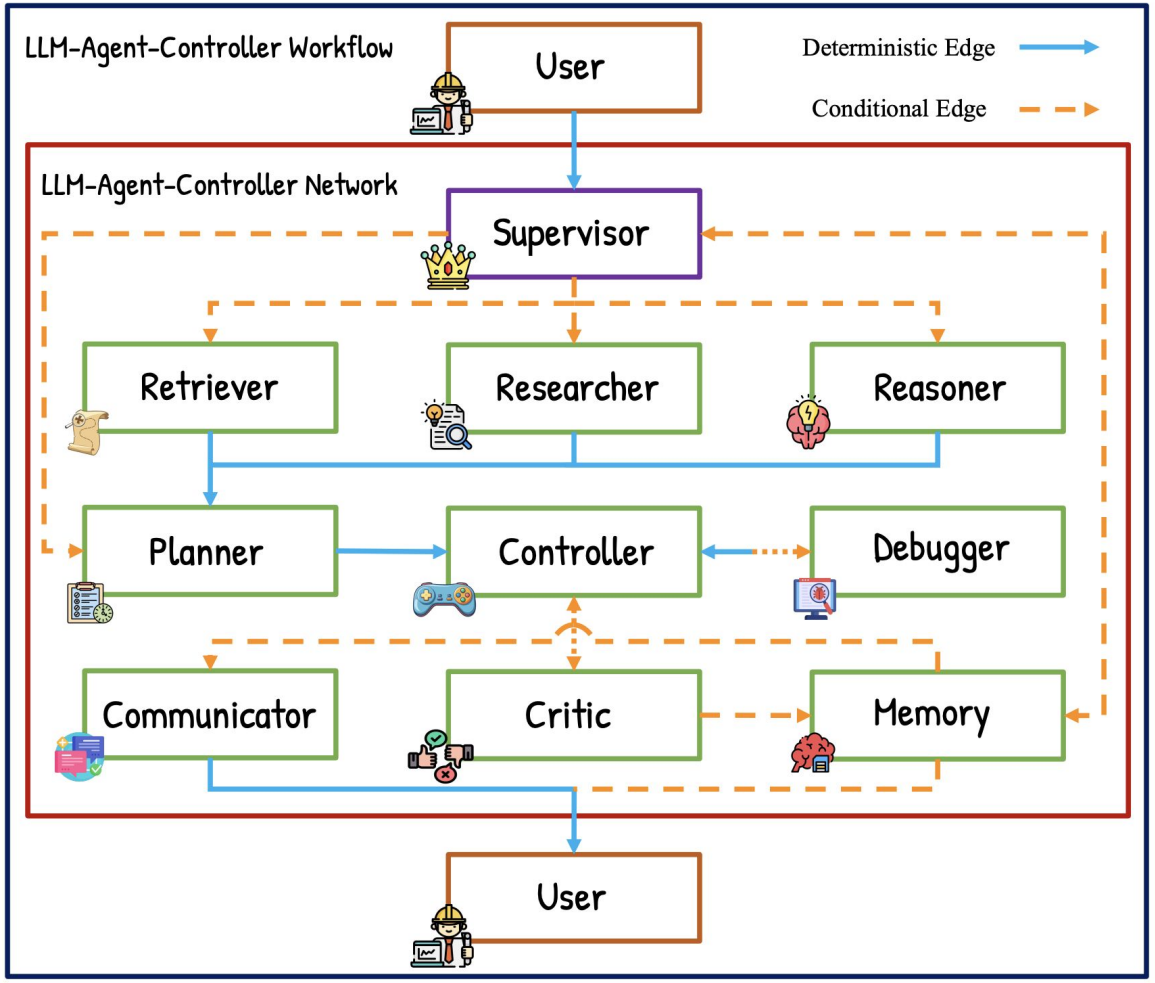
- 本システムは、中央のスーパーバイザー(Supervisor)の管理下で、その配下の専門エージェント(Retriever Agent、Planner Agent、Memory Agent、etc)が協調して様々なタスクを処理する。
- 専門エージェントの中核として、Controller Agent が制御工学のタスクを扱う。

LLM-Agent-Controller

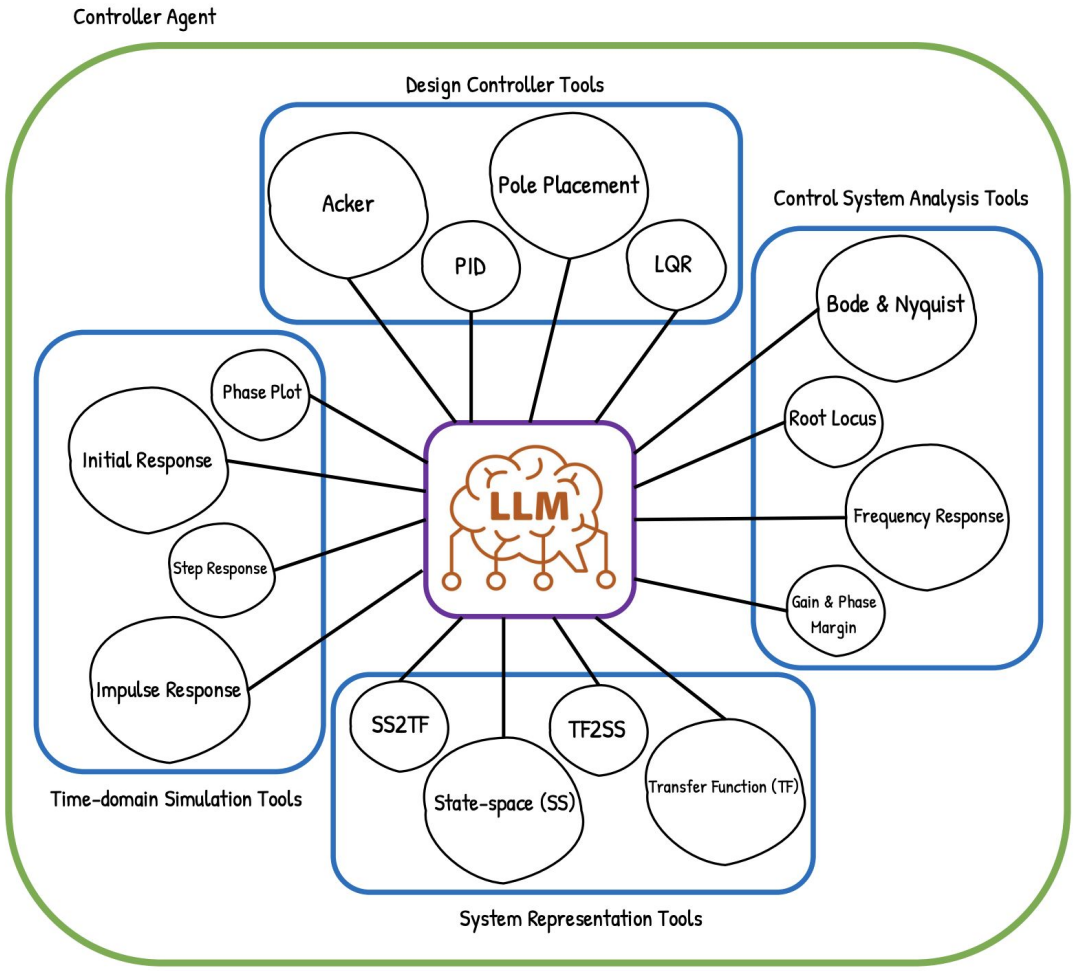
[手法]

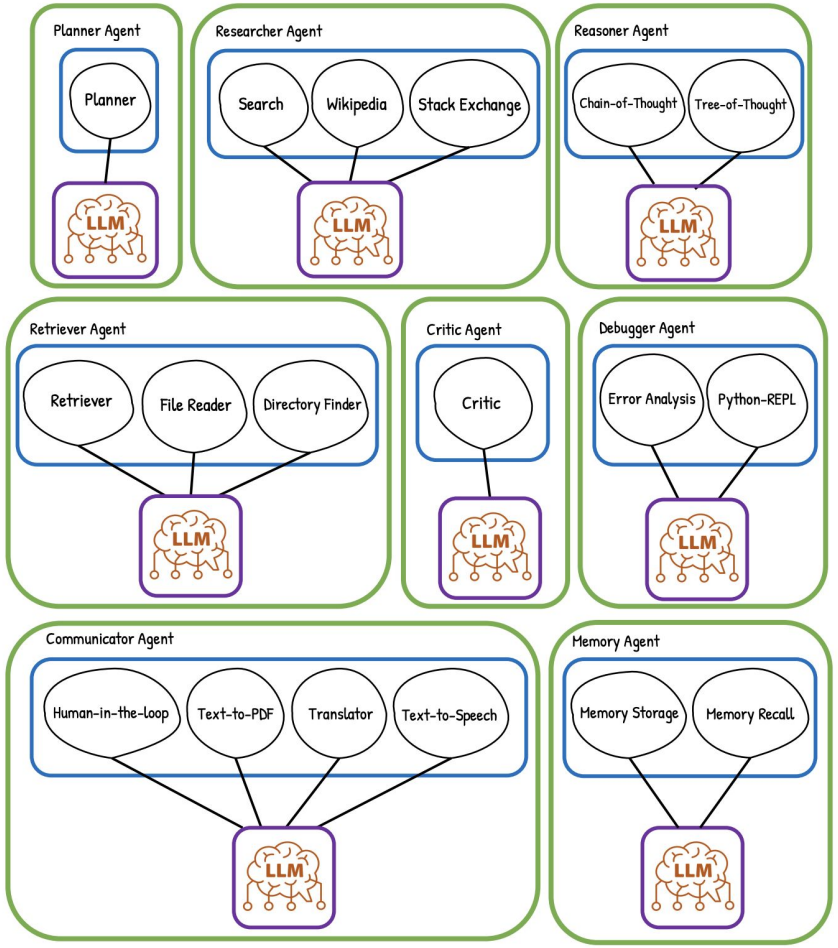
- ユーザーからの問い合わせを受信後、スーパーバイザーが高レベルの意思決定を行い、配下の専門エージェントを呼び出すかどうかを判断する。
- 情報を取得するエージェントとしては、Retriever Agent（RAGによる外部ソースからのデータ取得）や Research Agent（インターネット検索）がある。
- Planner Agent は、Controller Agent が使用するべきツールの実行順序を整理する。
- Controller Agent は、制御工学に関連した140種類以上のツールを備えている。
- Memory Agent は、短期メモリ及び長期メモリを効率的に管理する。

LLM-Agent-Controller



LLM-Agent-Controller





Cognitive Architectures for Language Agents

[概要]

- 汎用的なLLMエージェントを設計するための概念的枠組み「Cognitive Architecture for Language Agents（CoALA）」の提案。
- 20世紀前半には、数学分野をはじめとして、プロダクションシステムを導入する潮流があった。プロダクションシステムでは、規則を「前提条件」と「その条件が満たされた時に実行する行動」のペアとして定義する。
- 従来AIでは、あらかじめ多くの規則を定義する必要があった。それに対しLLMは、与えられた入力系列に対して、どのような出力系列が最適かという確率分布を学習しており、確率的なプロダクションシステムとして捉えることができる。
- 本論文では、LLMとプロダクションシステムの類似性に着目し、プロダクションシステムに用いられてきた認知アーキテクチャをLLMエージェントに適用する枠組みとして、CoALAを提案する。

Cognitive Architectures for Language Agents

$$
\begin{aligned}
(\text{temperature} > 70^\circ) \wedge (\text{temperature} < 72^\circ) &\rightarrow \text{stop} \\
\text{temperature} < 32^\circ &\rightarrow \text{call for repairs; turn on electric heater} \\
(\text{temperature} < 70^\circ) \wedge (\text{furnace off}) &\rightarrow \text{turn on furnace} \\
(\text{temperature} > 72^\circ) \wedge (\text{furnace on}) &\rightarrow \text{turn off furnace}
\end{aligned}
$$

Cognitive Architectures for Language Agents

[手法]

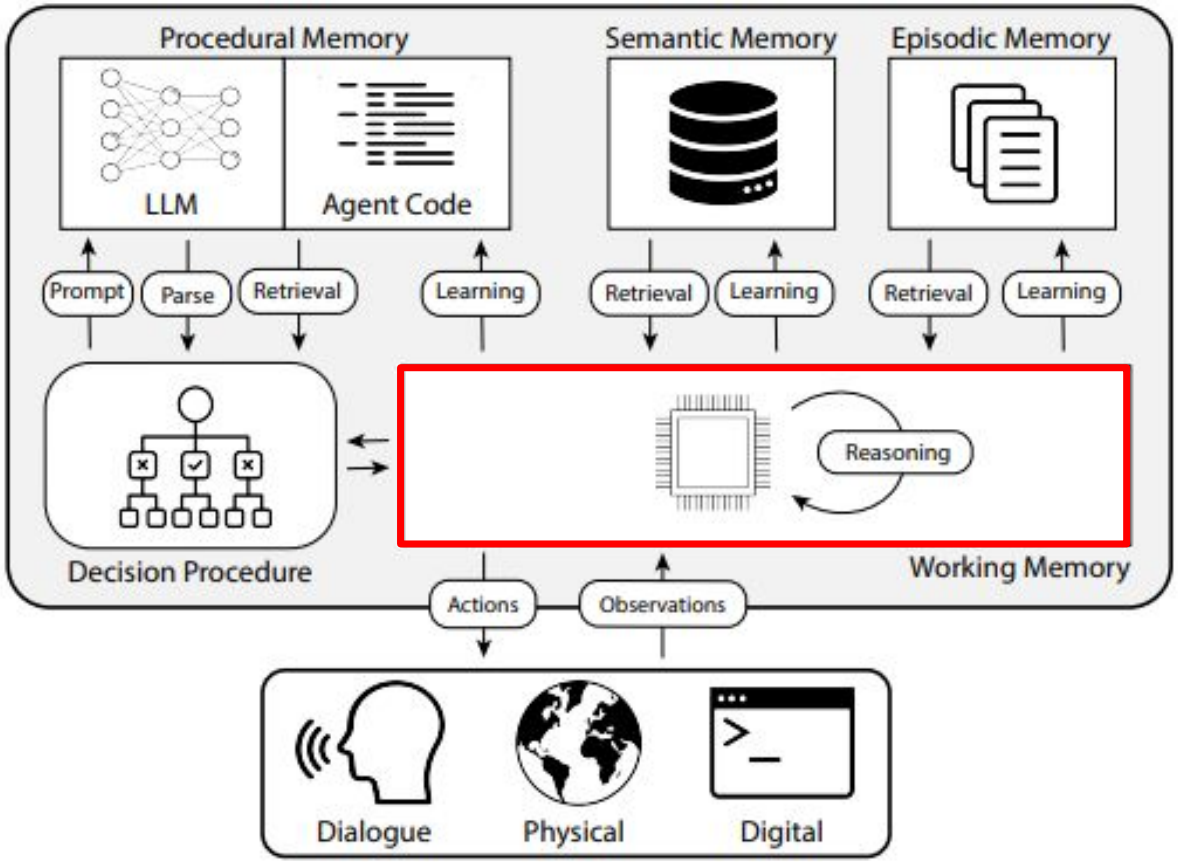
- CoALAは、LLMエージェントを主に3つのコンポーネントに整理する。
  ① メモリ機能（短期記憶・長期記憶）
  ② 行動空間（内部行動・外部行動）
  ③ 意思決定サイクル
- CoALAを用いて特定アプリケーション向けのLLMエージェントを開発するには、上記3つのコンポーネントを設計する必要がある。

Cognitive Architectures for Language Agents

[手法 - メモリ機能（短期記憶）]

- 論文中では、ワーキングメモリと記載されている。
- 現在の意思決定サイクルにおいて、LLMエージェントの状態を保持する。
  これには、最近の知覚入力、推論によって生成された知識、長期記憶から検索された知識、前回の意思決定サイクルから引き継がれた中核的な情報（LLMエージェントの目標など）が含まれる。
- 短期記憶の情報は、LLMに入力される。
- 短期記憶はLLMと長期記憶を繋ぐインタフェースとしても機能する。LLMエージェントの各構成要素を結びつける中心的なハブともいえる。

Cognitive Architectures for Language Agents



Cognitive Architectures for Language Agents

[手法 - メモリ機能（長期記憶）]

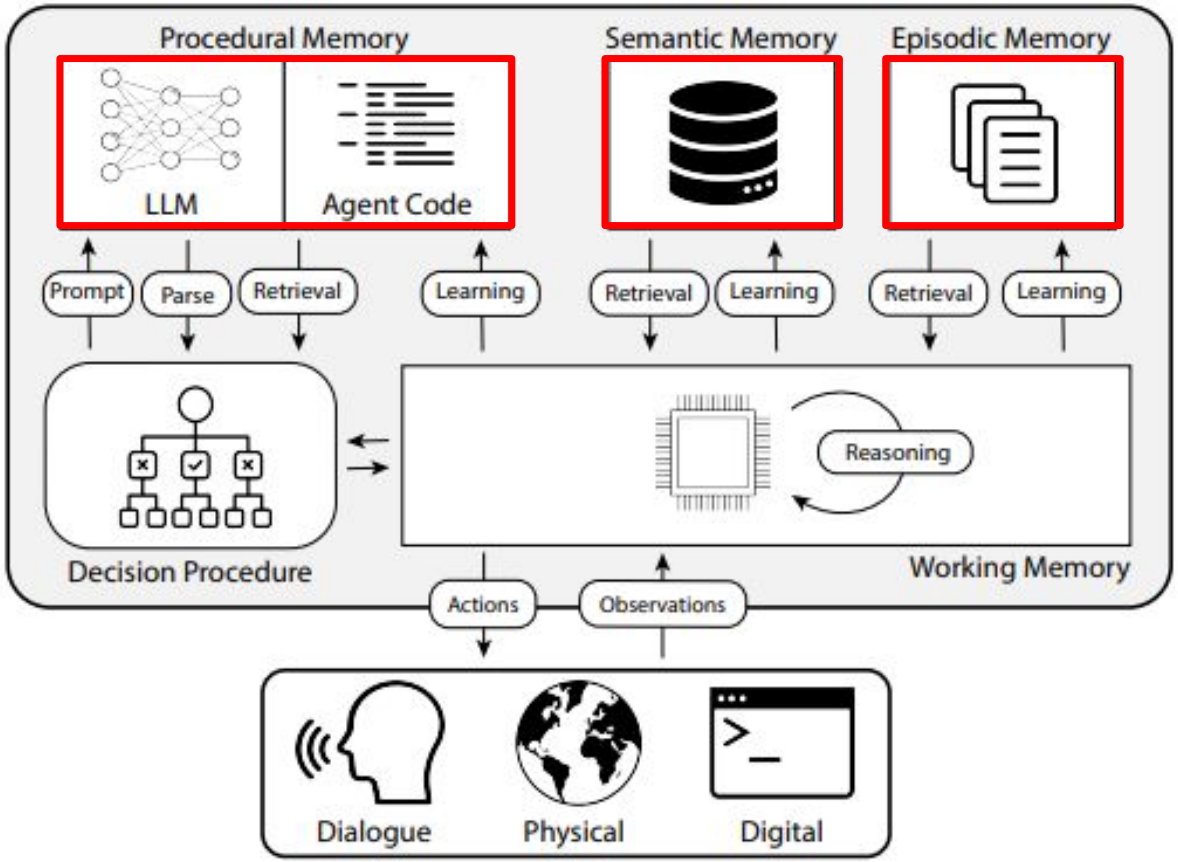
- 次の3種類に分けられる。
- エピソード記憶（Episodic Memory）：
  過去の意思決定サイクルにおける、LLMエージェントの行動系列を保持する。意思決定サイクルの計画段階では、エピソード記憶を短期記憶に抽出し、LLMの推論を支援することがある。
- 意味記憶（Semantic Memory）：
  世界に関する事実を保持する。

Cognitive Architectures for Language Agents

[手法 - メモリ機能（長期記憶）]

- 手続き記憶（Procedural Memory）：
  LLMの重みに保持された暗黙的知識（Implicit Knowledge）と、LLMエージェントのコードに記述された明示的知識（Explicit Knowledge）からなる。
  LLMの重み更新（ファインチューニング）は高コストであるため、LLMエージェントが稼働中に自律的に更新するのは現状では難しい。
  また、LLMエージェント自身がソースコードを更新することも可能だが、エピソード記憶や意味記憶への書き込みに比べて遥かにリスクが高い。バグの混入や、設計者の意図に反する挙動を招く恐れがある。

Cognitive Architectures for Language Agents

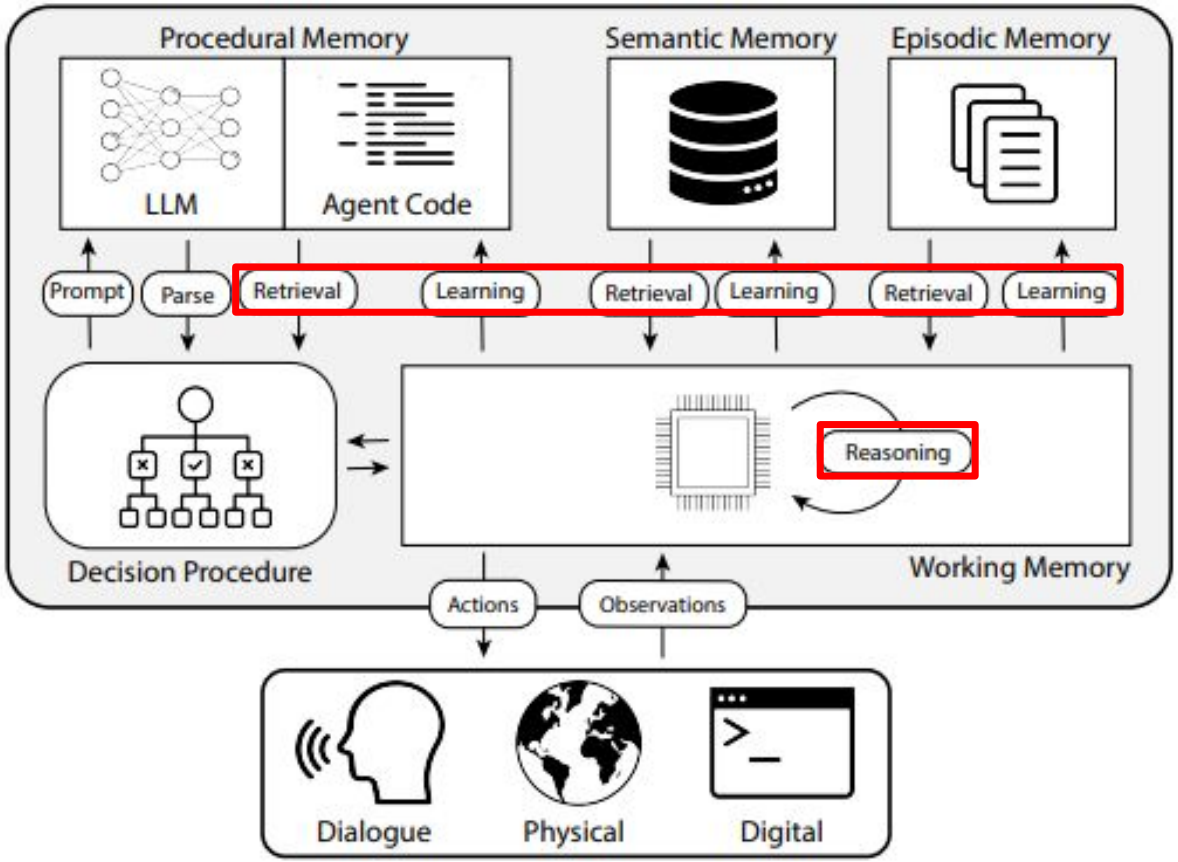


Cognitive Architectures for Language Agents

[手法 - 行動空間（内部行動）]

- 次の3種類に分けられる。なお、LLMエージェントを設計する際には、各メモリへのアクセス権限（読み取り・書き込み）をどう設定するかを決める必要がある。
- 検索：
  長期記憶から情報を短期記憶に読み出す。
- 推論：
  短期記憶の内容を処理し、新たな情報を生成する。短期記憶への書き込みも同時に行う。
- 学習：
  情報を長期記憶へ書き込む。

Cognitive Architectures for Language Agents

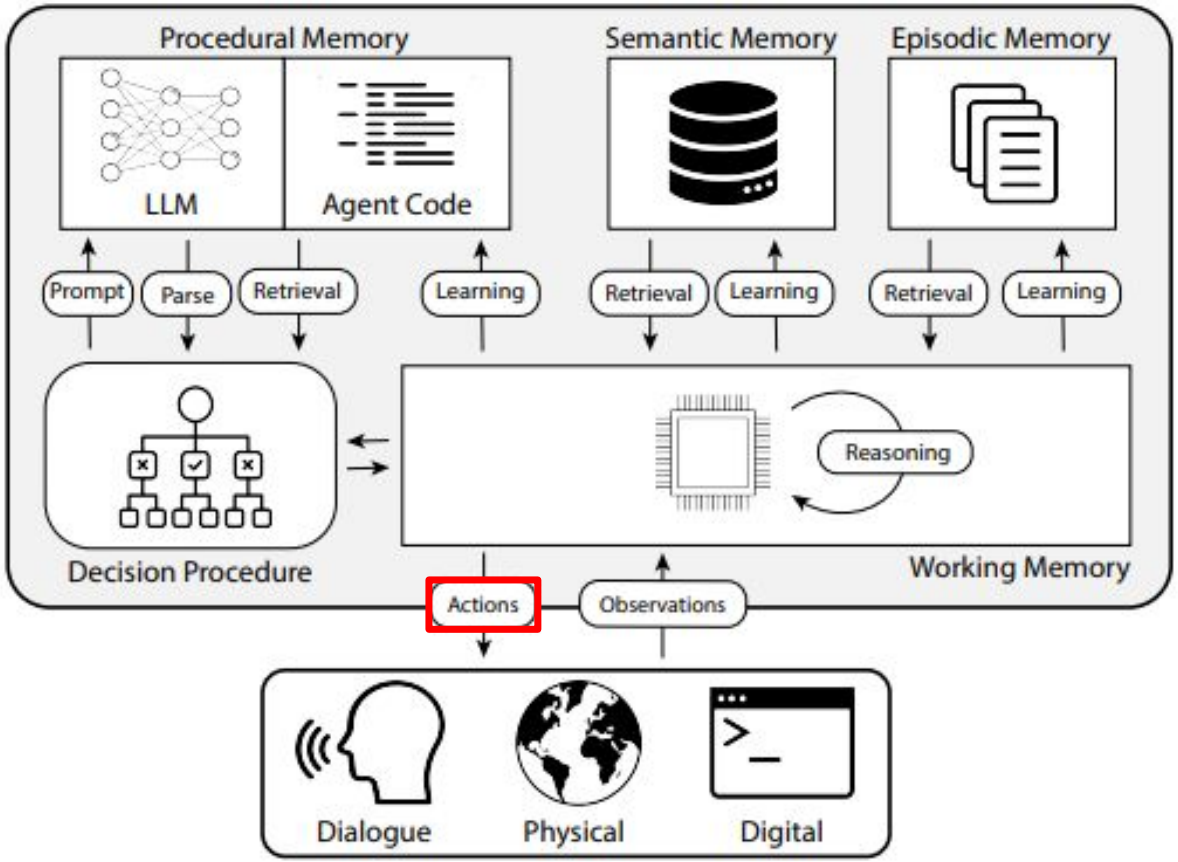


Cognitive Architectures for Language Agents

[手法 - 行動空間（外部行動）]

- 物理環境での行動に加え、デジタル環境におけるAPI呼び出し（検索、翻訳などのツール利用）も含まれる。

Cognitive Architectures for Language Agents

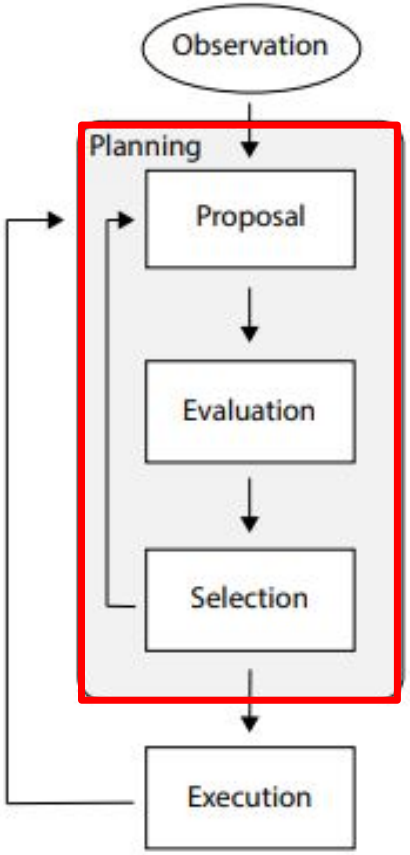


Cognitive Architectures for Language Agents

[手法 - 意思決定サイクル（計画段階）]

- 次の3種類に分けられる。推論や検索などの内部行動を柔軟に組み合わせながら、行動候補を生成・評価する。
- 提案：
  LLMが1つ以上の行動候補を生成する。通常は推論を用い、必要に応じて検索も行う。より高度なLLMエージェントでは、if-else や while-if のコードを用いる場合もある。
- 評価：
  各行動候補を定量的に評価する。この評価には、経験則的なルール↓LLMの評価指標（perplexity）、LLMによる推論などが用いられる。
- 選択：
  行動候補から1つを選択するか、全てを棄却して再び提案ステップに戻るかを決定する。

Cognitive Architectures for Language Agents

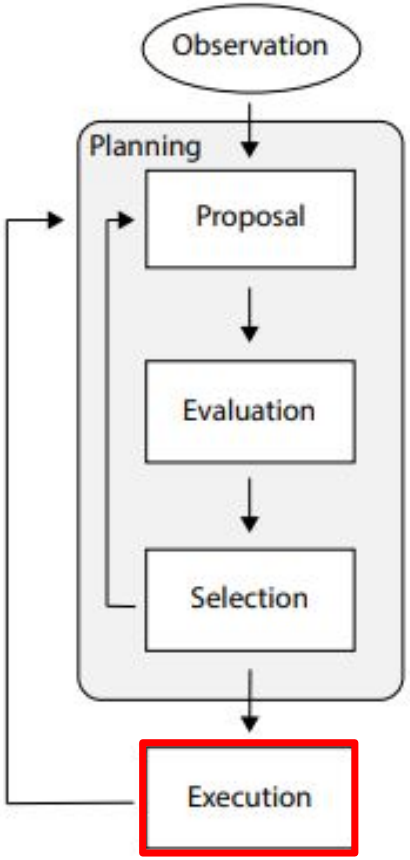


Cognitive Architectures for Language Agents

[手法 - 意思決定サイクル（実行段階）]

- 計画段階で選択された行動を実行する。
- 内部行動（学習）、あるいは外部行動（API呼び出しなど）を行う。

Cognitive Architectures for Language Agents

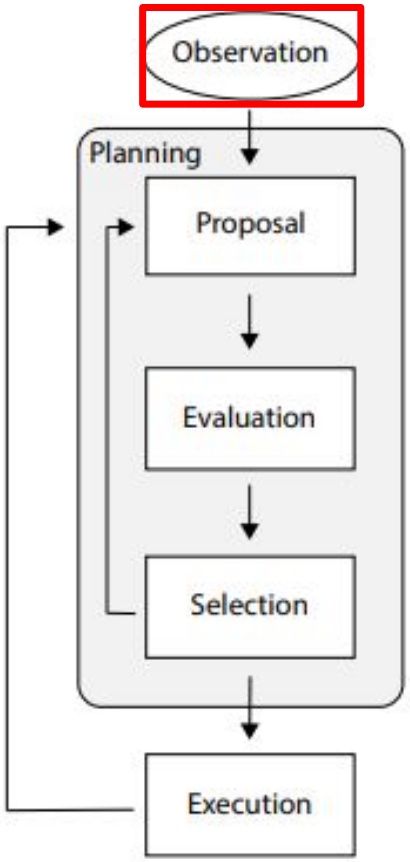


Cognitive Architectures for Language Agents

[手法 - 意思決定サイクル（観測段階）]

- 行動実行後、行動結果に対するフィードバックが与えられたうえで、次の意思決定サイクルへ進む。

Cognitive Architectures for Language Agents



claude-code

[概要 - KAIROS]

- assistant/sessionHistory.ts には、KAIROSと呼ばれるモードの実装がある。
- これは常駐するClaudeアシスタントで、ユーザーの入力なしに、監視やログファイルへの記録を行い、AIアシスタントが気付いた事柄に対して積極的に対応する。
- Ambient Agent の考え方に非常に似ている。
- 本システムには15秒の許容時間（timeout）が設定されている。AIアシスタントの介入が許容時間を超過するような操作については、全てpendingされる仕組みとなっている。
- AIアシスタントの応答を簡潔にし、メッセージでターミナルを占有しないようにするためのBriefモードも有効可能である。

claude-code

[概要 - ULTRAPLAN]

- 複雑な計画タスクをCloud Container Runtime (CCR) セッションにオフロードし、最大30分間の思考時間を与え、ブラウザから結果を承認できるようにするモード。
- 基本的なフローは以下の通り:
  1. Claude Code が、深い思考を要するタスクを特定する。
  2. 設定ファイルを介して、CCRセッションを起動する。
  3. 3秒間ごとに、ターミナルに思考状況を表示する。
     ブラウザベースのUIでは、計画の進捗状況を確認したり、結果を承認／却下することができる。
  4. 結果が承認されると、ターミナルに結果が転送される。

claude-code

※：プロンプト抜粋（原文は英語）

> 「あなたは夢を見ているのです。記憶のファイルをじっくりと見直す作業です。最近学んだことを、永続的で整理された記憶として統合することで、今後の学習セッションで素早く方向転換できるようになります。」

[概要 - autoDream]

- SubAgentとして動作するメモリ統合バックエンド。
- 以下条件を""全て""満たした場合に実行される：
  1. Time Gate：最後の動作から24時間経過している場合
  2. Session Gate：直近の動作から少なくとも5セッション経過している場合
  3. Lock Gate：（よく分らんかった）
- 実行後、autoDreamは事前定義されたプロンプト※（services/autoDream/consolidationPrompt.ts）に基づき、4つのフェーズを実施する。
  1. Memory.mdファイルから、内容を改善するべき既存トピックを特定する。
  2. 保持する価値のある新たな情報を探し出す。
  3. Memory.mdファイルを更新する。
  4. Memory.mdファイルを、200行以下＆25KB以下の状態にプルーニングする。

claude-code

[概要 - Coordinator Mode]

- coordinator/coordinatorMode.ts を介して起動されるマルチエージェントシステム。
- 機能フラグ（COORDINATOR_MODE）が有効になると、
  Claude Codeは単一エージェントから、複数のワーカーエージェントを並列に生成、指示（作業の続行及びストップ）するコーディネーターエージェントに変化する。
- ワーカーの生成及び指示は、コーディネーターのツールとして実装されている。

claude-code

※：システムプロンプトで250行程ある。詳細はソースを参照

[概要 - Coordinator Mode]

- コーディネーターに与えるシステムプロンプトは以下の通り※：
  1. コーディネーターの役割
  2. ツールの情報及びXMLメッセージフォーマットの説明
  3. タスクのワークフロー（Phases）の例示
  4. ワーカーに与えるプロンプトを作成する際の注意点（例示あり）
  5. やり取りしている様子の例示

段階	誰が	目的
研究	労働者（並列）	コードベースを調査し、ファイルを見つけ、問題を理解する
合成	コーディネーター	調査結果を読み、問題点を理解し、仕様書を作成する
実装	労働者	仕様ごとに的を絞った変更を行い、コミットする
検証	労働者	変更をテストすると正常に動作します

claude-code

[概要 - Coordinator Mode]

- ワーカーエージェント同士はXMLメッセージ（task-notification）を介して通信することができ、永続的な知識を共有するためのディレクトリ（shared scratchpad directory）もある。
- 本システムには、コンテキスト隔離のためのAgent Teams／Swarm機能や、メモリ同期機能などもある。

claude-code

[概要 - System Prompt Architecture]

- システムプロンプトは、事前定義された単一の文字列ではなく、キャッシュされたモジュール（constants/systemPromptSections.ts 等に実装あり）から実行時に構成する手法を取っている。
- システムプロンプトの各モジュールは、SYSTEM_PROMPT_DYNAMIC_BOUNDARYマーカーにより2種類に分割される。
  - 静的モジュール：ユーザーを跨いでキャッシュされる、不変のシステムプロンプト
  - 動的モジュール：あるユーザー（もしくはセッション）ごとにキャッシュされる、特有なシステムプロンプト。

claude-code

[概要 - Tools]

- tools/ では、40以上のツールが定義されている。
- getAllBaseTools()を介してツールが登録された後、ユーザー属性や環境フラグ、フォルダ権限設定などによってフィルタリングされる。

Tool	What It Does
AgentTool	Spawn child agents/subagents
BashTool / PowerShellTool	Shell execution (with optional sandboxing)
FileReadTool / FileEditTool / FileWriteTool	File operations
GlobTool / GrepTool	File search (uses native `bfs` / `ugrep` when available)
WebFetchTool / WebSearchTool / WebBrowserTool	Web access
NotebookEditTool	Jupyter notebook editing
SkillTool	Invoke user-defined skills
REPLTool	Interactive VM shell (bare mode)
LSPTool	Language Server Protocol communication
AskUserQuestionTool	Prompt user for input
EnterPlanModeTool / ExitPlanModeV2Tool	Plan mode control
BriefTool	Upload/summarize files to claude.ai
SendMessageTool / TeamCreateTool / TeamDeleteTool	Agent swarm management
TaskCreateTool / TaskGetTool / TaskListTool / TaskUpdateTool / TaskOutputTool / TaskStopTool	Background task management
TodoWriteTool	Write todos (legacy)
ListMcpResourcesTool / ReadMcpResourceTool	MCP resource access
SleepTool	Async delays
SnipTool	History snippet extraction
ToolSearchTool	Tool discovery
ListPeersTool	List peer agents (UDS inbox)
MonitorTool	Monitor MCP servers
EnterWorktreeTool / ExitWorktreeTool	Git worktree management
ScheduleCronTool	Schedule cron jobs
RemoteTriggerTool	Trigger remote agents
WorkflowTool	Execute workflow scripts
ConfigTool	Modify settings (internal only)
TungstenTool	Advanced features (internal only)
SendUserFile / PushNotification / SubscribePR	KAIROS-exclusive tools

claude-code

[概要 - compile-time feature flags]

- コンパイル時の機能フラグのONOFFにより、Claude Codeの各種機能の利用可否を設定している。(compile-time feature elimination)

Flag	What It Gates
`PROACTIVE` / `KAIROS`	Always-on assistant mode
`KAIROS_BRIEF`	Brief command
`BRIDGE_MODE`	Remote control via claude.ai
`DAEMON`	Background daemon mode
`VOICE_MODE`	Voice input
`WORKFLOW_SCRIPTS`	Workflow automation
`COORDINATOR_MODE`	Multi-agent orchestration
`TRANSCRIPT_CLASSIFIER`	AFK mode (ML auto-approval)
`BUDDY`	Companion pet system
`NATIVE_CLIENT_ATTESTATION`	Client attestation
`HISTORY_SNIP`	History snipping
`EXPERIMENTAL_SKILL_SEARCH`	Skill discovery

エージェント形態

AgentNet

[概要]

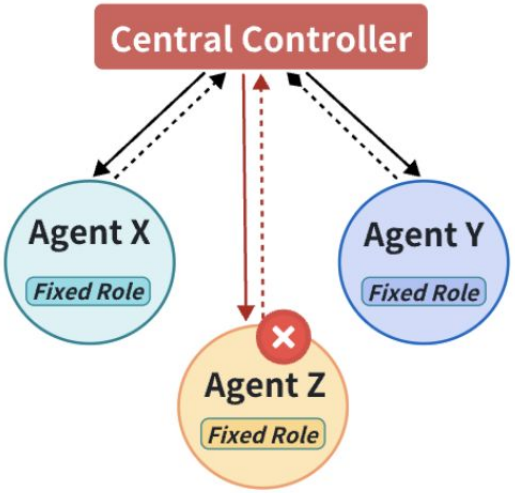
- 現在の主流である、マルチエージェントシステム（Multi-Agent System; MAS）の中央集権型アーキテクチャには、以下の課題がある。
  - 中央のコントローラーが単一障害点（Single Point of Failure; SPOF）となる。
  - エージェントの役割及びワークフローが静的であるため、タスク要件の複雑さに応じて動的にタスクを割り当てることができず、適応性に欠ける。
- 上記の課題を受け、本論文では、以下の特徴を持つ分散型アーキテクチャ「AgentNet」を提案する。
  - 単一障害点であったコントローラーを排除し、意思決定権を全てのエージェントに分散することで、フォールトトレランスが高まる。
  - RAGベースの記憶システムにより、過去の経験をもとに適応的に役割（専門性）を学習することが可能。また、リアルタイムに最適化されるネットワークトポロジーにより、ワークフローを柔軟に最適化できる。

AgentNet

Pre-Defined vs Self-Evolving Multi-Agent System

Pre-Defined Agents

hierarchical, static, single point of failure

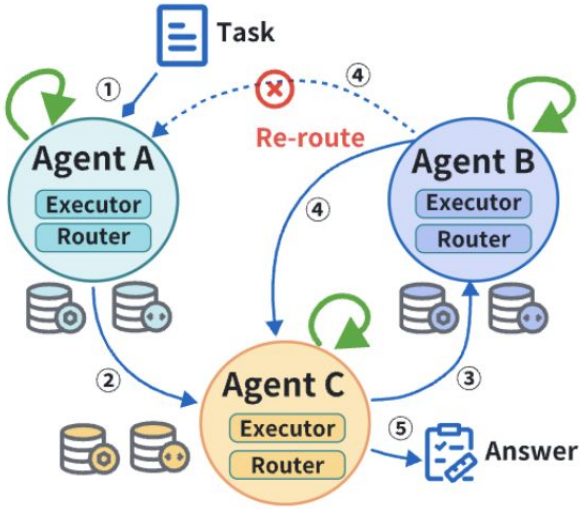


Key Features:

- Static role assignments
- Centralized task management
- Rigid workflows
- Single point of failure risk

Self-Evolving Agents (AgentNet)

Adaptive, decentralized, fault-tolerant



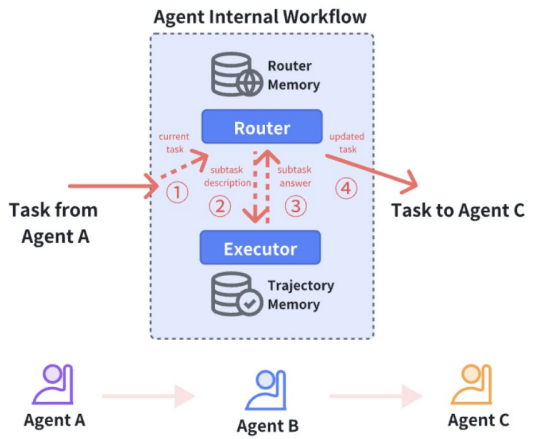
Key Features:

- Dynamic expertise development
- Autonomous task routing
- Experience-driven adaption
- Decentralized coordination

AgentNet

[手法]

- AgentNetの各エージェントは、ReAct（Reasoning and Acting）フレームワークに基づいており、主に2つのモジュールを内包している。
  - ルーター：クエリとコンテキスト（過去の経験）から、適切なアクションをLMが推論する。
  - エクゼキューター：ルーターでの推論結果及びコンテキスト（過去の経験）からLMが具体的なアクションに変換する。



AgentNet

[結果]

- 数学タスク、論理的質問応答タスク、及びAPI呼び出しタスクにおける正答率(Accuracy)の結果。
- MetaGPTなどの中央集権型MASを一貫して上回っている。

Backbone	Category	Method	MATH(Acc/%)	BBH(Acc/%)	API-Bank(Acc/%)
DeepSeek-V3	Single Agent	Direct	47.86	69.00	26.00
		React	77.14	88.00	29.00
		Synapse	89.28	92.00	28.00
		Self-Consistency	88.00	85.00	29.00
		Self-Refinement	87.14	84.00	25.00
	Multi-Agent	MorghAgent	39.29	56.00	16.00
		MetaGPT	92.14	64.00	22.00
		AFLOW	91.67	88.00	28.00
		GPTSwarm	72.14	90.00	21.00
		AgentNet	**92.86**	**94.00**	**30.00**
GPT-4o-mini	Single Agent	Direct	31.43	59.00	15.00
		React	55.71	80.00	24.00
		Synapse	77.14	79.00	22.00
		Self-Consistency	54.28	85.00	22.00
		Self-Refinement	68.57	81.00	23.00
	Multi-Agent	MorghAgent	80.71	56.00	16.00
		MetaGPT	73.57	53.00	19.00
		AFLOW	**85.00**	75.00	21.00
		GPTSwarm	**85.00**	**86.00**	13.00
		AgentNet	**85.00**	**86.00**	**29.00**
Qwen-turbo	Single Agent	Direct	37.85	57.00	27.00
		React	53.57	69.00	23.00
		Synapse	67.14	68.00	24.00
		Self-Consistency	64.28	70.00	28.00
		Self-Refinement	76.43	74.00	23.00
	Multi-Agent	MorghAgent	16.43	56.00	9.00
		MetaGPT	63.57	51.00	20.00
		AFLOW	**82.14**	57.00	22.00
		GPTSwarm	79.29	75.00	30.00
		AgentNet	81.43	**92.00**	**32.00**

AgentNet

[結果]

- エージェントの多様性がパフォーマンスに与える影響を調査した結果。
- 3エージェントの場合、LLMモデルと能力が同一（Fully Homogeneous）が最良のパフォーマンスを示した。小規模チームでは、均一な性能を持つエージェント同士の協調がより効果的であることを示唆している。
- 5エージェントの場合、LLMモデルと能力が同一（Fully Homogeneous）が最低のパフォーマンスを示した。大規模チームでは、LLMモデルまたは能力の多様性により、エージェントの協調性や相補的な推論能力が強化されることを示している。

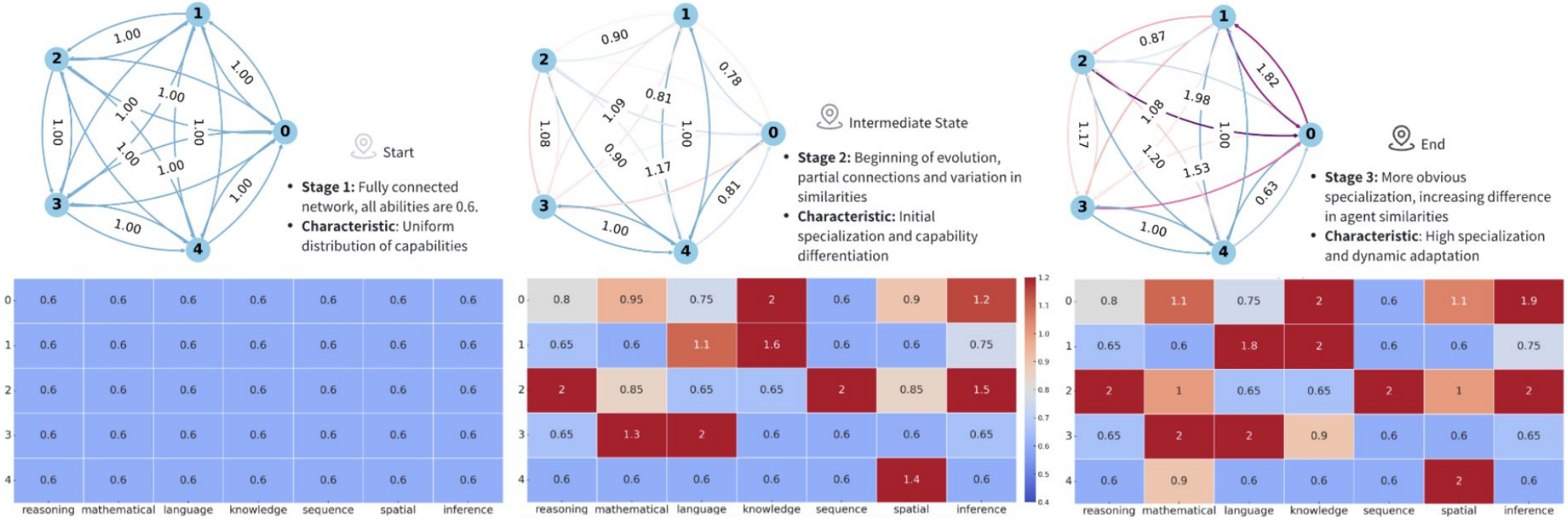
Setting	Fully Homogeneous	Skill Hetero.	LLM Hetero.	Both Hetero.
3 Agents	0.86	0.84	0.81	0.81
5 Agents	0.79	0.86	0.85	0.85

AgentNet

[結果]

- ネットワークトポロジーの進化過程。
- 初期状態（左図）では、ネットワークは全結合状態にある。この段階ではエージェントは専門家しておらず、全てのエージェントがタスクを処理する能力を等しく備えている。
- 中間状態（中央図）では、ネットワークが進化（重みが変動）するとともに、エージェントが専門家を始めている。重みが増している部分は、より緊密な連携を示している。
- 最終状態（右図）までに、ネットワークは最適化され、エージェントは明確な専門性を持っていることが分かる。

AgentNet


Start
Stage 1: Fully connected network, all abilities are 0.6.
Characteristic: Uniform distribution of capabilities
Intermediate State
Stage 2: Beginning of evolution, partial connections and variation in similarities
Characteristic: Initial specialization and capability differentiation
End
Stage 3: More obvious specialization, increasing difference in agent similarities
Characteristic: High specialization and dynamic adaptation
reasoning mathematical language knowledge sequence spatial inference

Building 17 Agentic AI Patterns and Their Role

- TBD

ソースコード

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L13]

- 状態グラフの状態（SupervisorState(TypedDict)）を定義する。
  - TypedDictはtypingライブラリに含まれるモジュールで、キーに格納されるデータの型をコード上で明示することが可能になる。[解説]
  - 状態の名称が SupervisorState となっているが、実際には他の専門エージェント（Researcher, Writer, etc）とも共有されているため注意。
  - messages: Annotated[list[BaseMessage], operator.add]
    - 会話履歴を表す。
    - Annotated[T, x] はtypingライブラリに含まれる型で、型Tそのものに対して任意のメタデータXを持たせることが可能になる。[解説]
    - つまり、messagesの型はlist[BaseMessage]で、そのメタデータは operator.addになる。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L13]

- next_agent: str
  - 次に動作するべきエージェントを表す。
- task_complete: bool
  - タスクが完了したかどうかのフラグを表す。
- final_response: str
  - タスクが完了した場合の、ユーザーに対する最終応答を表す。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L24]

- Supervisorが決定できる内容（RouteDecision(BaseModel)）を定義する。
  - BaseModelはpydanticライブラリに含まれるモジュールで、出力データの検証や型ヒントを提供する。
  - next: Literal["researcher", "writer", "critic", "FINISH"] = Field(description="Which agent to call next, or FINISH if done")
    - 次に呼び出すべきエージェントの種類を表す。
    - Literalはtypingライブラリに含まれる型で、取りうる型を制限することが可能になる。nextは指定された4つの文字列（researcher, writer, critic, FINISH）のみを受け付ける。[解説]
    - Fieldはpydanticライブラリに含まれる関数で、BaseModel型のより詳細な検証ルールを設定することが可能になる。[解説]

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L24]

- reasoning: str = Field(description="Why this agent was chosen")
  - エージェントを選択した理由を表す。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L34]

- SupervisorにいるLLMの出力を制限する。
  - supervisor_llm = llm.with_structured_output(RouteDecision)
    - .with_structured_output は langchain-coreライブラリ に含まれるメソッドで、引数で指定されたスキーマに基づいて、LLMの出力を構造化することが可能である。[公式Docs][解説]
    - 構造化出力の手法としては、以下の2種類がある。[公式Docs]
      ① .with_structured_outputメソッド
      ② create_agent関数の引数 response_format の指定
    - SupervisorにいるLLMの出力を、RouteDecisionで定義した型（next, reasoning）のみに制限している。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L37-L70]

- SupervisorにいるLLMのシステムプロンプトを定義し、推論させている。
  - def researcher(state: SupervisorState) -> dict:
    - StateとしてSupervisorStateを与えている。
  - system_prompt = """You are a supervisor managing a team of specialists:
    1. researcher - Gathers information and facts … (省略)
    - システムプロンプトで与えている情報としては、以下の通り。
      ① 役割（あなたはSupervisorとしてチームを纏める必要がある）
      ② 配下にいる専門エージェント（researcher, writer, critic）の役割
      ③ ワークフロー（呼び出すエージェントの順序、etc）

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L37-L70]

- SupervisorにいるLLMのシステムプロンプトを定義し、推論させている。
  - messages = [SystemMessage(content=system_prompt)] + state["messages"]
    - 会話履歴（state["messages"]）を加えて、最終的なシステムプロンプトとしている。
  - decision = supervisor_llm.invoke(messages)
    - .invokeメソッドによりLLMが推論している。
  - return { "next_agent": decision.next, "messages": [AIMessage(content=f"[Supervisor] Routing to {decision.next}: {decision.reasoning}")], }
    - LLMが生成した次の遷移先（decision.next）を、状態グラフのState（state["next_agent"]）に格納している。（.with_structured_outputメソッドにより、decisionはRouteDecision型に構造化されている）

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L73-L94]

- ResearcherにいるLLMのシステムプロンプトを定義し、推論させている。
  - def researcher(state: SupervisorState) -> dict:
    - Supervisorと同じStateをResearcherにも与えている。(ここは参考にするべきではない。与えるStateは選別するべきだと考える)
  - system = """You are a research specialist. Your job: - Gather relevant facts and information - Be thorough but concise - Cite sources when possible - Focus on what's most useful for the task"""
    - システムプロンプトで与えている情報としては、以下の通り。
      ① 役割(あなたはResearcherです)
      ② 仕事内容と注意点

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L73-L94]

- CriticにいるLLMのシステムプロンプトを定義し、推論させている。
  - def critic(state: SupervisorState) -> dict:
    - Supervisorと同じStateをCriticにも与えている。(ここは参考にするべきではない。与えるStateは選別するべきだと考える)
  - system = """You are a quality critic. Your job: - Review the content objectively - Provide specific, actionable feedback - If the work is good, say "APPROVED" and explain why - If it needs work, explain exactly what to improve"""
    - システムプロンプトで与えている情報としては、以下の通り。
      ① 役割(あなたはCriticです)
      ② 仕事内容と注意点

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L153-L157]

- Routerで行う処理を定義している。
  - if state.get("task_complete"): return "finalize" return state["next_agent"]
    - task_completeがTrueの場合には、Finalizeノードに遷移する。
      それ以外の場合は、state["next_agent"]ノード（＝LLMが生成した次の遷移先（supervisor関数のreturnを参照））に遷移する。
    - .add_conditional_edges メソッドの第二引数で、Routerの処理を規定する必要がある。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L160-196]

- 状態グラフを構築している。
  - graph = StateGraph(SupervisorState)
    - StateGraph は langgraphライブラリに含まれるクラス。公式Docsの説明を見ると「A graph whose nodes communicate by reading and writing to a shared state.」とある。
    - グラフ構造は状態としてSupervisorState を保持している。
  - graph.add_node("supervisor", supervisor)
    - .add_node(node, action, etc) は langgraphライブラリに含まれるメソッドで、node にノード名[str]を、actionに 関数 or Runnbaleオブジェクト を指定する。[公式Docs]

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L160-196]

- graph.add_conditional_edges("supervisor", route_to_agent, { "researcher": "researcher", "writer": "writer", "critic": "critic", "finalize": "finalize", },)
  - .add_conditional_edges(source, path, path_map) は langgraphライブラリに含まれるメソッドで、sourceに条件付エッジを始めるノード名[str]、pathに次に遷移するノードを決めるための関数、path_mapに遷移できるノードの一覧[dict] を指定する。[公式Docs]
- return graph.compile()
  - .compileメソッドは langgraph に含まれるメソッドで、コンパイルされた実行可能なグラフ（CompiledStateGraph）を生成する。[公式Docs]
  - CompiledStateGraph は Pregel を継承しているため、継承元の .invoke メソッドなどを使用できる。[公式Docs][解説]

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L199-L235]

- 本プログラムの実行部分
  - agent = build_multi_agent_system()
    - build_multi_agent_system の return（＝CompiledStateGraph）が agent に格納される。
  - initial_state = { "messages": [HumanMessage(content="Write a short blog post about the benefits of AI in healthcare")], "next_agent": "", "task_complete": False, "final_response": "", }
    - 状態グラフ（SupervisorState）の初期状態を定義している。

GPUプログラミング

GPUを支える技術

[概要 - P.191]

- NVIDIAは、GPUを動かす機会命令を公開していない。その代わりに、PTX（Parallel Thread Excution）というアセンブラレベルの命令セットを公開している。
- GPuプログラミングを行う場合、アセンブラレベルのPTXを使用することはほとんどない。多くの場合、C言語をGPU向けに拡張したCUDAやOpenCLという言語が使われる。
-